

Exactly-Once Effects for Re-Planning LLM Agents: A Reduction, a Retention Hazard, and an Evidence Contract

Research-team working draft

Version 0.3 — 19 September 2026

Development results; not submitted; authorship confirmation pending

Abstract—A recovering language-model agent may re-express an authorized task with different tool calls. The runtime must decide which authorized effect occurrences remain available, not whether the new request text equals the old one. We study a restricted language of independent, explicitly authorized effect obligations executed against a durable sink. Three results follow. (i) *Reduction*. With a sink that atomically retains a per-key effect record for the whole recovery period, verified plan refinement reduces to stable obligation-key execution: an independent re-implementation reproduces the v0.2 null result exactly across 186,674 exhaustively enumerated regrouping cases, 240 real SIGKILL/recovery trials and 24 three-worker races, and a reference that also consults its own ledger ties the verifier call-for-call. (ii) *Retention hazard*. Commercial idempotency contracts are finite: Stripe prunes idempotency keys after 24 hours and treats a reused key as a new request; Amazon SQS FIFO deduplicates for 5 minutes. We model retention-bounded sinks $P_D(T)$, prove that same-key retry is safe exactly when the controller’s own hold timestamp is within the window, and show that every same-key-retry policy—including the strong reference and the v0.2 verifier—duplicates in all 149,217 lost-acknowledgement cases once the window closes, with a seven-step model-checking witness and 20/20 runtime duplicates for naive recovery versus 0/20 for the retention-aware controller. Completion after the window then depends on an evidence contract: the retention-aware controller completed 10/10 lost-acknowledgement trials with a lookup endpoint and left 10/10 unresolved without one. (iii) *Evidence contract*. A ternary per-obligation projection is sufficient for the maximal safe fresh-dispatch set but not for reconciliation cost: retaining the batch-semantics class of a lost request changes the exact optimal expected number of item probes from n (independent entries) to 1 (atomic), to about $\log_2(n+1)$ (ordered stop-on-failure), and to a closed form for reported cardinality, all matched by an exact adaptive oracle to $n = 10$ and reproduced probe-for-probe by a journal-driven recovery worker on the runtime. A takeover study across three provider profiles shows where fence, evidence and unresolved states are each necessary. No live LLM, commercial provider call, universal exactly-once guarantee or superiority over published systems is claimed.

Index Terms—agent systems, effect identity, crash recovery, idempotency, retention, authorization, partial observability

I. INTRODUCTION

An agent instructed to send one deployment notification to each of two recipients can use one batch call, two single-recipient calls, or a different ordering. After a crash it may choose another representation. Asking whether the new request has the same text as the old one does not specify the recovery

decision. The runtime must determine which authorized *occurrences* are being attempted and what is known about their previous effects.

Three ambiguities must be separated. Identical payloads can belong to different approvals: a user may legitimately request the same notification again. One approval can produce different request representations without authorizing additional effects. And a request may have committed even though its result is absent from the checkpoint. Existing work addresses parts of this space: semantic rollback attacks and replay-or-fork recovery [1], durable authorization instances bound to canonical actions [2], reservations under ambiguous provider outcomes [3], and task-level staged effects [4]. Durable-execution engines recommend that side-effecting steps be idempotent and restart each attempt from its initial state [7]; completion-record systems established durable request identity and atomic effect/result recording [5].

The v0.2 study began from the hypothesis that preserving effect obligations across plan changes would add safety or completion beyond a strong durable-workflow reference, and found a null result. This version asks the follow-on question that the null result forces: *under which provider contracts does stable-key execution stop being sufficient, and what must a controller then know?* Two answers turn out to be concrete and testable. First, the sink assumption behind the reduction—retention of the key/effect record throughout recovery—is not what commercial providers document. Stripe removes idempotency keys after they are at least 24 hours old and “generate[s] a new request if a key is reused after the original is pruned” [8]; Amazon SQS FIFO deduplication lasts five minutes [9]. A recovery that lands after the window is a retry into a sink that has forgotten the key. Second, when the sink no longer deduplicates, safe completion depends on outcome evidence, whose cost depends on what the controller’s journal retained about the lost request.

Contributions:

- 1) An independent re-implementation of the v0.2 kernel from its specification, reproducing every finite-domain count (Table I) and the process-crash and concurrency results, and closing the shared-code threat noted in v0.2. The reference is strengthened to consult its own ledger (R^+); it ties the verifier exactly.

- 2) A retention-bounded profile $P_D(T)$ with a necessary-and-sufficient condition for safe same-key retry that depends only on the controller’s hold timestamp and the documented window (Prop. 3); an exhaustive study showing all same-key policies fail after the window while the retention-aware controller does not (Table II); a new model-checking mutant with a seven-step witness; and a runtime study with a finite-TTL sink (Table V).
- 3) A takeover study in which a successor worker adopts a held occurrence while the crashed worker’s request is still in flight, across P_D , P_F and P_O with naive and profile-aware recovery (Table IV). It exhibits the three necessary mechanisms—sink dedup, fence-then-fresh-key, and an explicit *unresolved* state—as distinct rows rather than as prose.
- 4) A semantics-determined belief construction: recording the membership, order and processing class of a lost batch request determines the belief family exactly; ternary projection discards it. Closed forms for the atomic, prefix and exact-cardinality classes are verified against an exact adaptive oracle to $n = 10$ (Table VI), and a recovery worker that rebuilds the belief family from a one-field-per-request journal reproduces the expected probe counts exactly on the runtime while remaining exactly-once (Table VII).
- 5) A measured negative finding about a common evaluation design: three-worker races resolved by controller reservation never reach the sink’s deduplication path (0 of 96 sink calls), so they do not test the sink contract.

The empirical scope remains explicit. Proposals are generated by combinatorial transformations, not by a live LLM. The provider performs synthetic database insertions. Finite enumerations are exhaustive only in the stated models; process schedules are sampled. Commercial contract facts are quoted from official documentation, not exercised against a live account.

II. BACKGROUND AND PRIOR-ART BOUNDARY

A. Durability and completion records

RIFL retains completion records with operations and addresses retry routing and record lifetime [5]. A persistent identifier alone is insufficient if the external mutation and the deduplication record can diverge, if retries reach non-coordinating sinks, or if the record is reclaimed. RIFL treats record lifetime as a system-controlled parameter; a commercial provider controls it unilaterally, which is the gap this version studies.

CapLease binds authorization instances to canonical actions with durable budgets and transactional transitions; its Server Ledger reference prevents duplicate effects with an idempotent sink [2]. A result against regenerated keys therefore does not establish an advantage over that design; we do not execute CapLease.

B. Agent-specific execution boundaries

ACRFence analyzes regenerated calls and either reuses an old result or requires a fork [1]. AID-Guard holds a reservation while an outcome is ambiguous and permits replacement only under terminal-result or no-effect/fencing conditions [3]; our fence-then-fresh-key path follows that boundary. Cordon introduces task-level semantic transactions with result lineage and staged effects [4]; we do not claim all-or-nothing commit across obligations. Proof-carrying agent actions associate verifiable evidence with proposals [6]; our checker is an exact contract check, not a certificate system.

C. Durable execution and commercial idempotency

Temporal documents that an Activity “be idempotent, so retries can be processed without duplicate side effects” and that “each attempt starts from the initial state” [7]. Deduplication is thus delegated to the effect provider. Stripe saves the result of the first request under an idempotency key, compares incoming parameters to the original and errors on mismatch, and removes keys after at least 24 hours, after which a reused key produces a new request [8]. SQS FIFO deduplicates by `MessageDeduplicationId` within a 5-minute window and continues tracking the identifier after delivery [9]. These are the only two documented windows we rely on; both are finite.

D. Partial outcomes and evidence contracts

SQS `SendMessageBatch` reports each entry individually and can mix successes and failures under HTTP 200 [10]; SES `SendBulkEmail` returns “one object per intended recipient” [11]. These are *independent* per-entry semantics. Slack’s `conversations.history` provides message evidence after the fact, but for some application classes it is rate limited to one request per minute with at most 15 objects [12]; and Slack’s write method documents that on `fatal_error` “it’s possible some aspect of the operation succeeded before the error was raised” [13], which is the opaque-outcome ambiguity in a vendor’s own words. Planning with partial information and sensing is established [14]; our belief families are an application of that view.

III. PROBLEM MODEL

A. Three identities

An immutable approval contract is $C = (\iota, p, \alpha, \Omega, s, v)$: intent-instance identifier, principal, authenticated approval event, finite set of obligation occurrence identifiers, approved effect specification per obligation, and schema version. An *attempt* is a proposed or transmitted implementation of one or more obligations. An obligation identifies an occurrence, not a payload equivalence class; two identical notifications approved twice are two occurrences. The trusted approval layer creates ι and Ω ; re-presenting the same (p, α) recovers the same contract.

The executable language supports exact tuples (kind, target, body) with kind `notify`, a singleton wrapper and a batch

wrapper. Aliases, free-text inference, decomposition, cross-provider equivalence and state-dependent transformation are excluded. Obligations are independent.

Proposition 1 (Content-only ambiguity). *A decision procedure observing only old and new payloads cannot always distinguish replay under one approval from an identical action under a new approval.*

Proof. Completed x , lost receipt, repeated proposal yields observed input (x, x) ; completed x followed by a new approval for x yields the same input. The required decisions differ. Approval provenance is the missing observation. $\square$

B. Failure and trust assumptions

The proposer may reorder, regroup, omit, duplicate or alter proposed entries. The verifier rejects unauthorized entries before dispatching any prefix. Workers stop abruptly; concurrent workers may attempt the same obligations; a successor may adopt a held occurrence while the predecessor’s request is still in flight. Conversation rollback never rolls back the contract, the control ledger or the provider ledger. Excluded: malicious host, corruption of both stores, lost disks, broken database atomicity, hostile clock manipulation beyond a bounded skew.

C. Provider profiles, now with retention

Definition 1 (Profiles). $P_D(T)$: atomic per-key effect/result recording with immutable key/payload binding, retained for at least T after the first commit under the key; after T the key may be reclaimed and a request under it may commit again. $P_D = P_D(T)$ with $T = \infty$. P_F : no key deduplication, but a fence operation that prevents any later delivery under a key and returns a trustworthy final outcome for it. P_O : neither; lookups are point-in-time only.

Stripe’s documented behaviour is $P_D(T)$ with $T \geq 24$ h and immutable parameter binding [8]; SQS FIFO is $P_D(T)$ with $T = 5$ min [9]. No profile is silently promoted to another; in particular a negative lookup alone never establishes P_F .

IV. VERIFIED REFINEMENT AND ITS REDUCTION

A. Plan checking and stable keys

A plan contains an instance, schema and a sequence of wrapper groups. Verification checks the whole proposal: certified wrappers, legal arity, each slot approved and used at most once, and exact match of kind, target and body against the contract. Executable bytes are reconstructed from the contract. Partial plans are accepted; completed occurrences in a recovered plan are residualized and return cached completion.

The effect key is $K(\iota, \omega) = H(\text{obligation-key-v1}, \iota, \omega)$, independent of grouping and attempt identity. Each occurrence has durable state *ready* $\rightarrow$ *held* $\rightarrow$ *done*; this version adds a terminal *unresolved* state for a held occurrence that can no longer be retried safely and for which no finality evidence exists. A control transaction reserves the occurrence (with a lease and, for $P_D(T)$, a retention deadline $h(\omega) + T$) before transport.

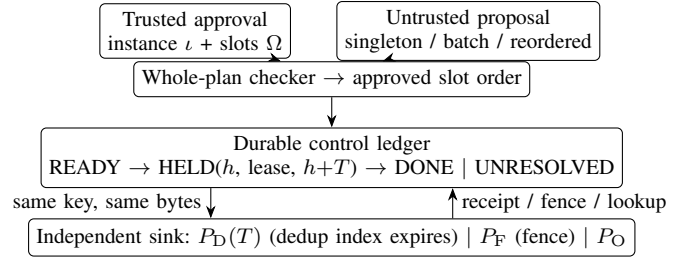


Fig. 1. Implemented boundary. The sink effect is a synthetic database insertion. Recovery of a HELD occurrence consults $h(\omega)$, T and the profile before choosing same-key retry, fence, evidence lookup or UNRESOLVED.

Proposition 2 (Stable-key reduction). *Assume immutable contracts, complete plan checking, mediated effect paths, an injective persistent key map, and $P_D(T)$ with every retry of a key completing before $h(\omega) + T$. Then any accepted sequence of regrouped plans and retries has at most one committed effect per approved occurrence and no effect outside the contract.*

Proof. Every dispatched effect derives from an approved occurrence and its stored specification. All attempts for that occurrence use the same key. While the dedup record is retained, atomic key/effect recording allows at most one effect per key and rejects payload substitution. The retention hypothesis guarantees retention at every retry. Distinct occurrences are not collapsed by key identity. Induction over sink operations preserves the invariant. $\square$

Corollary 1. *A reference that performs canonical-action matching, consults the same ledger and uses the same keys and sink contract reaches the same final effect-count vector and the same number of sink calls as verified residualization.*

The v0.2 reference R resent every occurrence and let the sink deduplicate, so residualization saved calls without changing effects. The strengthened reference R^+ consults its own *done* records first; Table I shows it ties V in calls as well. Verified refinement adds no exactly-once guarantee under this hypothesis; the corollary supplies the null hypothesis for every comparison below.

V. FINITE RETENTION

A. The hazard

The hypothesis “every retry completes before $h(\omega) + T$ ” is the load-bearing assumption of Prop. 2, and it is a property of the recovery *schedule*, not of the code. A worker crash followed by an outage longer than T —or a checkpoint restored days later—violates it. The controller then holds an occurrence in state *held* whose effect may or may not have committed; the sink has forgotten the key; and a same-key retry is indistinguishable, at the sink, from a first request.

Proposition 3 (Retention-bounded same-key retry). *Let the controller record $h(\omega)$ durably before transport, let clock skew and the delivery delay of the retried request be bounded by δ , and let the sink be $P_D(T)$. A same-key retry issued at*

controller time t is duplicate-free in every history consistent with the controller's knowledge if and only if $t < h(\omega) + T - \delta$.

Proof. (Sufficiency) Any earlier commit under the key occurred at some sink time $c \geq h(\omega) - \delta'$ for the skew component $\delta' \leq \delta$; its record is retained until at least $c + T \geq h(\omega) + T - \delta'$. The retry reaches the sink by $t + \delta''$ with $\delta' + \delta'' \leq \delta$, hence before the record can be reclaimed, and is deduplicated. (Necessity) If $t \geq h(\omega) + T - \delta$, the history in which the first request committed immediately at $c = h(\omega) - \delta'$ and its record was reclaimed at exactly $c + T$ is consistent with a lost acknowledgement; the retry arrives at or after reclamation and commits a second effect. $\square$

The decision uses only the controller's own ledger and a contract parameter T . No sink query is needed to make it, and no sink query can substitute for it: a negative lookup after reclamation is consistent with both histories.

Corollary 2 (Profile downgrade). *After $h(\omega) + T - \delta$, a $P_D(T)$ sink is, for that key, at best P_F (if it exposes finality or fence plus evidence) and otherwise P_O . A retention-aware controller is the stable-key controller plus this downgrade: positive evidence confirms; a fence permits a fresh key; absent both, the occurrence becomes unresolved.*

Where the objects a request created remain retrievable after the key is pruned—as Stripe's list endpoints permit for created objects, a capability we did not exercise here—the sink degrades to a lookup-capable profile; SQS offers no message lookup by deduplication identifier, so it degrades to P_O . The controller must therefore carry T and the evidence capability as explicit provider-contract fields.

B. Ambiguous replacement

A fresh key is different from a same-key retry. Certifying absence from a negative lookup and then dispatching a fresh key permits two distinct keys to commit one obligation if the old request can still arrive. Safe replacement in P_F needs an operation that certifies no old effect and prevents old delivery [3]; Section IX-D exercises this with a real in-flight request.

VI. OUTCOME KNOWLEDGE BEYOND INDEPENDENT STATES

A. Safe fresh-dispatch frontier

Under P_F , let a nonempty belief family $\mathcal{B} \subseteq 2^\Omega$ contain every finalized committed set consistent with sound evidence. Define $P(\mathcal{B}) = \bigcup \mathcal{B}$, $D(\mathcal{B}) = \bigcap \mathcal{B}$, $F(\mathcal{B}) = \Omega \setminus P(\mathcal{B})$ and $U(\mathcal{B}) = P(\mathcal{B}) \setminus D(\mathcal{B})$.

Proposition 4 (Maximal safe fresh dispatch). *A once-only fresh dispatch set S is duplicate-free in every world of $\mathcal{B}$ iff $S \subseteq F(\mathcal{B})$.*

Proposition 5 (No-observation completion boundary). *A single fixed fresh dispatch set both avoids duplicates and completes every obligation in every world of $\mathcal{B}$ iff $|\mathcal{B}| = 1$.*

Proofs are as in v0.2 and are exhaustively checked in E1. Both depend on fresh mutation, no new observation and old-dispatch finality; a P_D sink within its window can replay retained keys without identifying the completed set.

B. Safety sufficiency is not query sufficiency

The ternary projection (done on D , absent on F , unknown on U) determines F exactly, so correlations never enlarge the immediate safe frontier. They change the cost of learning more. The v0.2 witness $\mathcal{B}_1 = \{\{a\}, \{b\}\}$ versus $\mathcal{B}_2 = 2^{\{a,b\}}$ has identical projections and optimal probe costs one and two.

C. Semantics-determined belief families

The source of correlation in practice is not exotic: it is the processing semantics of the request whose response was lost. Let a request carry ordered members $M = (\omega_1, \dots, \omega_n)$ and let the journal record M , its order and the provider's documented processing class.

Definition 2 (Classes). *independent: any subset may have committed ($\mathcal{B} = 2^M$; per-entry outcomes as in SQS batch and SES bulk [10], [11]). atomic: all or nothing ($\mathcal{B} = \{\emptyset, M\}$; a single-object request such as one Stripe charge [8]). prefix: ordered processing that stops at the first failure ($\mathcal{B} = \{\{\omega_1.. \omega_k\} : 0 \leq k \leq n\}$; the sequential worker of this artifact executing a batch wrapper is one instance). exact- k : a trustworthy aggregate reports k successes ($\mathcal{B} = \binom{M}{k}$).*

Proposition 6 (Journal sufficiency and probe costs). *Recording $(M, \text{order}, \text{class})$ determines $\mathcal{B}$ exactly. Every class projects to all-unknown on M , so a ternary journal believes 2^M and needs $|M| = n$ unit probes. Under a uniform prior over worlds, the exact optimal adaptive expected probe cost is n (independent), 1 (atomic), $\lceil \log_2 N \rceil - \frac{2^{\lceil \log_2 N \rceil} - N}{N}$ with $N = n+1$ (prefix), and $n - \frac{n-k}{k+1} - \frac{k}{n-k+1}$ (exact- k).*

Proof. Determination is by construction. Independent: each probe resolves one item and items are independent, so n probes are necessary and sufficient. Atomic: any single probe resolves the world. Prefix: worlds are the N prefixes; probing ω_i splits them into prefixes of length $\geq i$ and $< i$, so any adaptive policy is a binary decision tree with N uniformly likely leaves and expected depth minimized by the complete tree, whose leaves lie at depths $\lfloor \log_2 N \rfloor$ and $\lceil \log_2 N \rceil$ with $2^{\lceil \log_2 N \rceil} - N$ leaves at the shallower depth. Exact- k : the v0.2 gap-symmetry argument; the count-aware stopping rule is optimal by exchangeability. $\square$

The class is a property the controller already reads from provider documentation to choose a profile; retaining it per request costs one field. The claim is about information discarded by the ternary abstraction, not an inability of every ternary-journal implementation to be extended.

VII. IMPLEMENTATION AND REPRODUCIBILITY

The v0.3 artifact is a fresh implementation written from the v0.2 manuscript rather than from its source tree, in Python standard library plus SQLite. Controller and provider run in

different processes with different database files, WAL mode and `synchronous=FULL`. The controller stores immutable contracts, key/payload bindings, occurrence states with lease and retention deadline, receipts and events; insertion is transactional and (p, α) is unique. The provider inserts one row per effect atomically; a $P_D(T)$ dedup row carries an expiry; a P_F fence table is consulted inside the effect transaction so an in-flight request that arrives after the fence is rejected; every `/execute` call is logged with its outcome so attempts are distinguishable from effects. An evidence endpoint searches the retained effect table, modelling providers whose history outlives their idempotency window; it can be disabled.

Two policies share transport and persistence. R performs flat canonical-action matching and consults the ledger per occurrence; V checks typed wrapper membership on the whole plan and residualizes. Two recovery modes exist for a held occurrence: *naive* (same-key retry regardless of profile or retention) and *aware* (Cor. 2). For the evidence study the worker can send a wrapper group as one multi-entry request; the provider processes entries by a declared class (independent, atomic, prefix) with a per-entry failure hook, and the controller journals membership, order and class before transport. A recovering worker with a *ternary* journal probes every held member; with a *structured* journal it rebuilds the belief family of Section VI and probes by the exact oracle. Fault points terminate the process with a real SIGKILL after the first reservation, after the first effect before confirmation, after the first confirmation, after the third effect, or during the first transport (a timer kills the process 150 ms after the request is sent while the provider delays commit).

The artifact contains 29 kernel unit tests, all experiment drivers, raw per-trial records (472 process trials with return codes, crash markers, successor summaries and provider-side counts), exhaustive enumerations and model-checking witnesses. `run_all.py` regenerates every number in this paper in about ten minutes on one core. Manifests are local provenance, not external timestamping.

VIII. STUDY DESIGN

D1 and D1-TTL. For $n = 1..4$, enumerate all $n!2^{n-1}$ ordered groupings as source and destination and every committed prefix of the source order: $\sum_{n=1}^4 (n!2^{n-1})^2(n+1) = 186,674$ cases. Policies: G (group-scoped key), R , R^+ , V , V_r (retention-aware, no evidence), V_r+E (retention-aware with lookup). Six variants cross the state of the last committed occurrence at the crash—durably confirmed, committed with lost acknowledgement, or reserved without effect—with whether the dedup index was reclaimed before recovery.

M1. One obligation, two key generations, two senders per generation, one crash, plus a retention-expiry transition that fires only while no request is in flight. Breadth-first search exhausts the safe configuration; each mutant stops at its shortest violating trace. State counts are specific to this encoding and not comparable to v0.2's.

I1, I2. As in v0.2: $2 \times 6 \times 4 \times 5 = 240$ SIGKILL/recovery trials on P_D ; 12 three-worker races per policy.

TABLE I

D1: EXHAUSTIVE REGROUPING ON P_D WITH UNBOUNDED RETENTION. DUPLICATES ARE CASE COUNTS, NOT AGENT FAILURE RATES. R^+ AND V HAVE IDENTICAL EFFECT VECTORS IN EVERY CASE AND IDENTICAL CALL COUNTS.

n	Plans	Cases	G dup	R dup	R^+ dup	V dup
1	1	2	0	0	0	0
2	4	48	16	0	0	0
3	24	2,304	1,308	0	0	0
4	192	184,320	129,024	0	0	0
Total		186,674	130,348	0	0	0
Sink calls			744,290	744,290	372,145	372,145

I3. Source plan one batch of four; the worker is killed 150 ms into its first transport while the provider delays commit by 1000 ms (*early*) or 200 ms (*late*). A different worker adopts the held occurrence after lease expiry at 0.5 s or 0.9 s. 3 profiles $\times$ 2 variants $\times$ 2 recovery modes $\times$ 5 repetitions = 60 trials.

I4. $P_D(T)$ with $T = 0.5$ s. The worker is killed after the first effect (lost acknowledgement) or after the first reservation (no effect); the successor starts 0.8 s later. 2 evidence settings $\times$ 2 faults $\times$ 2 recovery modes $\times$ 2 policies $\times$ 5 = 80 trials.

I5. One batch request over n members is sent and the worker is killed before it can read the response; the provider fails the entries designated by the world. Every world of each class is run once: independent $n=4$ (16 worlds), atomic $n=4, 8$ (2 each), prefix $n=4, 8$ (5 and 9). The successor starts after the provider finished (finality by construction), reconciles from a ternary or a structured journal, confirms committed members and fresh-dispatches absent ones under new keys on P_F . 68 trials.

E1–E4. E1 enumerates all 65,535 belief families on four occurrences against all 16 dispatch sets. E2 is the correlation witness. E3 compares the exact- k closed form with an exact dynamic-programming oracle for all $0 < k < n \leq 10$ (2,026 worlds; v0.2 stopped the oracle at $n = 8$). E4 computes ternary and structured optimal probe costs for the independent, atomic and prefix classes.

The success oracle for every process trial requires the crashed worker to exit with -9 , the successor with 0, and reads the provider's effect table independently of controller state.

IX. RESULTS

A. The reduction holds and the strengthened reference ties

Table I reproduces the v0.2 counts exactly (16 / 1,308 / 129,024 group-key duplicates) from an independent implementation. R^+ and V agree on every effect vector and make the same 372,145 calls; R makes twice as many because it resends completed occurrences and lets the sink deduplicate. In I1 all 240 trials pass for both policies; the provider logs 1,080 execute calls for 960 effects, and the 120 excess calls are exactly the lost-acknowledgement retries (60 trials each at the after-first-effect and after-third-effect fault points) answered with a cached receipt. In I2 all 24 races pass; the provider

TABLE II

D1-TTL: THE SAME 186,674 CASES WHEN THE DEDUP INDEX IS RECLAIMED BETWEEN CRASH AND RECOVERY. *dup*: CASES WITH A DUPLICATED OCCURRENCE; *unres*: CASES THE CONTROLLER LEAVES UNRESOLVED; *inc*: CASES WHERE AN APPROVED OCCURRENCE NEVER HAPPENED.

Last occ. at crash	Policy	dup	unres	inc	calls
confirmed	R	149,217	0	0	744,290
	R^+, V	0	0	0	372,145
	V_r, V_r+E	0	0	0	372,145
lost ack	R	149,217	0	0	744,290
	R^+, V	149,217	0	0	521,362
	V_r	0	149,217	0	372,145
	V_r+E	0	0	0	521,362
reserved, no effect	R	111,760	0	0	744,290
	R^+, V	0	0	0	521,362
	V_r	0	149,217	149,217	372,145
	V_r+E	0	0	0	670,579

TABLE III

M1 BOUNDED REACHABILITY. ONLY SAFE ROWS EXHAUST THEIR GRAPH; MUTANTS STOP AT THE SHORTEST WITNESS.

Model	Configuration	States	Edges	Witness
v0.2	safe	1,180	3,467	none (exhausted)
	absence without fence	445	1,036	9 (duplicate)
	fresh-key retry	395	939	7 (duplicate)
	missing as success	11	11	2 (false success)
	no sink dedup	65	126	6 (duplicate)
+retention	safe	1,396	4,043	none (exhausted)
	retry after expiry	121	249	7 (duplicate)

logs 96 calls, all committed, none deduplicated—controller reservation resolves every race before the sink is reached. Median successor wall time is 0.101 s (R) and 0.102 s (V) including interpreter start; it is descriptive only.

B. After the window, every same-key policy duplicates

Table II is the central new result. With reclaimed retention and a lost acknowledgement, R , R^+ and V all duplicate in every one of the 149,217 cases with a nonempty committed prefix. The verifier’s plan-level residualization protects only occurrences the controller already recorded as done. The retention-aware policy is safe in every variant; without evidence it pays with 149,217 unresolved cases, which are false alarms under lost acknowledgement and real gaps when nothing had committed. With a lookup it is safe and complete everywhere, at the cost of one evidence call per lapsed held occurrence. Without retention expiry, all six policies except G are safe and complete in all three crash-state variants.

C. Model checking

The retention mutant’s witness is: hold, send, commit, crash, retention expires, resend same key, commit. The safe configuration with the retention transition still exhausts without violation because its send rule refuses any key that may have been dispatched before expiry; recovery proceeds through fence, lookup and certificate instead.

TABLE IV

I3: A SUCCESSOR ADOPTS A HELD OCCURRENCE WHILE THE CRASHED WORKER’S REQUEST IS STILL IN FLIGHT (*early*) OR AFTER IT COMMITTED (*late*). CELLS ARE COUNTS OUT OF 5. *dup*: PROVIDER RECORDED TWO EFFECTS; *done*: CONTROLLER FINISHED ALL FOUR; *unres*: CONTROLLER LEFT ONE UNRESOLVED.

Profile	Variant	Recovery	dup	done	unres	Mechanism
P_D	early	naive	0	5	0	sink dedup
P_D	early	aware	0	5	0	sink dedup
P_D	late	naive	0	5	0	cached receipt
P_D	late	aware	0	5	0	cached receipt
P_F	early	naive	5	5	0	none
P_F	early	aware	0	5	0	fence rejects old, fresh key
P_F	late	naive	5	5	0	none
P_F	late	aware	0	5	0	fence returns receipt
P_O	early	naive	5	5	0	none
P_O	early	aware	0	0	5	unresolved (world done)
P_O	late	naive	5	5	0	none
P_O	late	aware	0	5	0	positive lookup

TABLE V

I4: RECOVERY 0.8 s AFTER A CRASH ON A $P_D(T)$ SINK WITH $T = 0.5$ s. COUNTS OUT OF 5; R AND V ARE IDENTICAL IN EVERY CELL AND ARE MERGED.

Evidence	Crash point	Recovery	dup	done	unres
lookup	after first effect	naive	5	5	0
lookup	after first effect	aware	0	5	0
lookup	after reservation	naive	0	5	0
lookup	after reservation	aware	0	0	5
none	after first effect	naive	5	5	0
none	after first effect	aware	0	0	5
none	after reservation	naive	0	5	0
none	after reservation	aware	0	0	5

D. Takeover with an in-flight request

Table IV separates the three mechanisms. Under P_D the sink absorbs the race in both orders. Under P_F naive retry duplicates every time; the fence either rejects the in-flight request and licenses a fresh key or returns the committed receipt. Under P_O the only safe early action is to stop: the successor’s lookup is negative because the old request has not landed yet, and 0.5 s later the world is in fact complete while the controller correctly reports one unresolved occurrence. The naive controller “completes” every P_F and P_O trial with a duplicate. The controller never learns which happened without the provider’s effect table, which is why the oracle reads it directly.

E. Retention expiry at runtime

Naive same-key retry after the window duplicates in 20 of 20 lost-acknowledgement trials, for both policies. The retention-aware controller never duplicates. With a lookup it completes all lost-acknowledgement trials; without one, or when nothing had committed, it stops at *unresolved* because a negative point-in-time lookup is not finality evidence. The “after reservation, aware, lookup” row is the price of correctness under $P_D(T)$ without a fence: the fresh dispatch would have been safe, but the controller cannot know it.

TABLE VI

E4: EXACT OPTIMAL EXPECTED ITEM PROBES FOR A LOST REQUEST OVER n MEMBERS, UNIFORM PRIOR. TERNARY JOURNAL COST IS n FOR EVERY CLASS.

n	Ternary	Atomic	Prefix (exp.)	Prefix (worst)	Exact- $\lfloor n/2 \rfloor$
2	2	1	1.667	2	1.000
4	4	1	2.400	3	2.667
6	6	1	2.857	3	4.500
8	8	1	3.222	4	6.400
10	10	1	3.545	4	8.333

TABLE VII

I5: RUNTIME RECONCILIATION OF A LOST MULTI-ENTRY RESPONSE, EVERY WORLD OF EACH CLASS RUN ONCE. ALL 68 TRIALS ARE EXACTLY-ONCE AND COMPLETE. MEAN PROBES ARE MEASURED LOOKUPS; EXPECTATION IS THE E4 ORACLE.

Class	n	Journal	Worlds	Mean	Max	Oracle
indep.	4	ternary	16	4.00	4	4.00
indep.	4	structured	16	4.00	4	4.00
atomic	4	ternary	2	4.00	4	4.00
atomic	4	structured	2	1.00	1	1.00
prefix	4	ternary	5	4.00	4	4.00
prefix	4	structured	5	2.40	3	2.40
prefix	8	ternary	9	8.00	8	8.00
prefix	8	structured	9	3.22	4	3.22
atomic	8	ternary	2	8.00	8	8.00
atomic	8	structured	2	1.00	1	1.00

F. Evidence contracts

E1 performs 1,048,560 checks with no mismatch against the frontier rule or the singleton characterization; 941 of 65,535 families admit a nonempty safe dispatch. E2 confirms costs one and two for identical ternary snapshots. E3 agrees with the exact- k closed form in all 45 (n, k) pairs and 2,026 worlds, and the exact oracle matches it for every $n \leq 10$. Table VI shows the DP oracle and the closed forms agreeing for every n . The prefix class is the strongest realistic case for the journal contract: with $n = 8$ the structured journal needs 3.2 probes on average and 4 in the worst case, against 8 for the ternary journal. The atomic class needs one. These are expected counts of unit-cost synthetic queries; a provider that returns all item statuses in one call at cost c_L makes the structured journal worthwhile only when c_L exceeds the corresponding gap.

Table VII closes the loop between the mathematics and the implementation: the structured journal’s measured probe counts equal the oracle expectation in every cell, the ternary journal pays n in every cell, and both are exactly-once and complete in all 68 trials because absent members are dispatched under fresh keys only after the world is identified. The journal costs one field per request. Finality of the lost request is guaranteed here by timing; on a real P_F provider it must come from the fence.

X. DISCUSSION AND THREATS TO VALIDITY

A. What changed

The v0.2 null result stands and is now reproduced by a second implementation with a stronger reference. The contri-

bution is no longer “verified refinement” but the precise boundary of the reduction and what lies beyond it. Two boundary conditions are documented provider facts, not constructions: finite retention windows and vendor-acknowledged opaque outcomes. On the retention side the necessary controller knowledge is minimal and already durable ($h(\omega)$ and T), and the correct behaviour after the window is a profile downgrade, not a heuristic. On the evidence side the necessary knowledge is one field per request (the processing class), and its value is exact and class-dependent.

For an agent runtime this means: a tool adapter must declare T , the evidence capability and the batch class as contract fields; a controller that stores only “unknown outcome” and never retries is safe under P_O but forfeits completion that a lookup would have restored; and a controller that retries with the same idempotency key without checking $h(\omega) + T$ is not exactly-once against Stripe or SQS FIFO after an outage longer than the window.

B. Internal and construct validity

The same process designed policies, workloads and oracles; the provider’s independent effect table reduces but does not remove design bias. Reference and verifier share storage and transport; a common bug could affect both, which is why the exhaustive abstract study exists alongside the runtime. The finite model abstracts torn writes and packet corruption. Exactly-once is measured at the synthetic insertion boundary. The P_F fence and the evidence endpoint are our own implementations of documented capabilities, not vendor endpoints. I3 and I4 depend on sleeps and are deterministic in this environment across 5 repetitions per cell; they are demonstrations of mechanism, not measurements of frequency.

C. External and statistical validity

There are no live LLM traces; the frequency with which agents regroup or replan is unmeasured. Retention windows are quoted from documentation current on the access date and can change. The prefix class is grounded in this artifact’s own sequential worker and in ordered stop-on-failure processors generally; we did not verify a specific vendor batch API with that semantics. Exhaustive counts do not justify confidence intervals about deployments; we report exact counts only.

D. Evidence and authorship boundaries

All numbers derive from executed programs in the artifact. No paid inference, production mutation or public submission occurred. Generative-AI assistance was used for code, mathematics and text; human review of proofs, citations, code and authorship is required before submission. Characterizations of [1], [2], [3], [4], [6] are carried from the v0.2 literature audit and must be re-verified against the current versions of those preprints.

XI. NEXT EVALUATION GATE

The retention hazard is now specific enough to test against a real contract without spending on effects: a sandbox account

for a $P_D(T)$ provider can confirm the documented post-window behaviour with test-mode objects, and the controller field set $(T, \text{evidence}, \text{class})$ can be validated against three adapters with different profiles. The evidence-contract claim needs one provider with a per-request processing class other than independent, and a comparison that grants the structured journal’s information to a strong baseline. Natural LLM proposals should be collected separately from stress transformations. If both gates pass, the runtime contract fields become the deliverable; if they fail, this remains a reproducibility and negative-result study with a documented hazard.

XII. CONCLUSION

Stable obligation keys are sufficient for exactly-once effects of re-planning agents exactly as long as the sink retains them, and an independent re-implementation confirms that a verifier adds nothing under that hypothesis. Commercial sinks bound retention. Beyond the bound, the same policies duplicate in every lost-acknowledgement case, the safe decision depends only on the controller’s own hold time and the documented window, and completion depends on an evidence contract whose cost is determined by what the journal retained about the lost request. The artifact supplies exact finite-domain, model-checked and process-level evidence for each of these statements and states the conditions under which each stops applying.

REFERENCES

- [1] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, “ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore,” arXiv:2603.20625, 2026.
- [2] J. Xu, L. Fan, Z. Wang, X. Li, and H. Liu, “Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions,” arXiv:2608.01710, 2026.
- [3] Y. Tong, L. Dai, and S. Guo, “AID-Guard: Stateful authorization for delegated agent effects,” arXiv:2608.21159, 2026.
- [4] Z. Chen et al., “Cordon: Semantic transactions for tool-using LLM agents,” arXiv:2606.17573, 2026.
- [5] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, “Implementing linearizability at large scale and low latency,” in *Proc. SOSP*, 2015, pp. 71–86.
- [6] Z. Wang, “Proof-carrying agent actions: Model-agnostic runtime governance for heterogeneous agent systems,” arXiv:2606.04104, 2026.
- [7] Temporal Technologies, “What is a Temporal Activity?” Official documentation, accessed 19 September 2026. <https://docs.temporal.io/activities>
- [8] Stripe, “Idempotent requests,” API reference, accessed 19 September 2026. https://docs.stripe.com/api/idempotent_requests
- [9] Amazon Web Services, “Using the message deduplication ID in Amazon SQS,” Developer Guide, accessed 19 September 2026. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/using-messagededuplicationid-property.html>
- [10] Amazon Web Services, “SendMessageBatch—Amazon SQS API Reference,” accessed 19 September 2026. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/APIReference/API_SendMessageBatch.html
- [11] Amazon Web Services, “SendBulkEmail—Amazon SES API v2 Reference,” accessed 19 September 2026. https://docs.aws.amazon.com/ses/latest/APIReference-V2/API_SendBulkEmail.html
- [12] Slack, “conversations.history,” Web API method reference, accessed 19 September 2026. <https://api.slack.com/methods/conversations.history>
- [13] Slack, “chat.postMessage,” Web API method reference, accessed 19 September 2026. <https://api.slack.com/methods/chat.postMessage>
- [14] R. I. Brafman and G. Shani, “Replanning in domains with partial information and sensing actions,” *J. Artif. Intell. Res.*, vol. 45, pp. 565–600, 2012.
- [15] SQLite, “Atomic commit in SQLite” and “Write-ahead logging,” official documentation, accessed 19 September 2026. <https://sqlite.org/atomiccommit.html>